Agno Framework - Complete Beginner's Guide

A Comprehensive Documentation with Examples and Usage Instructions

---

Table of Contents

---

1. Introduction to Agno

What is Agno?

Agno is a **multi-agent framework, runtime, and control plane** built for speed, privacy, and scale. It's designed to help you build production-ready AI applications with minimal code.

Key Features

Category	Feature	Description
Core Intelligence	Model Agnostic	Works with 40+ LLM providers (OpenAI, Anthropic, Google, AWS, etc.)
	Type Safe	Enforce structured I/O through `input_schema` and `output_schema`
	Dynamic Context	Inject variables, state, and data on the fly
Memory & Knowledge	Persistent Storage	Session history, state, and messages in databases
	User Memory	Recall user-specific context across sessions
	Agentic RAG	Connect to 20+ vector stores with hybrid search
Execution & Control	Human-in-the-Loop	Native support for confirmations and manual overrides
	Guardrails	Built-in validation, security, prompt protection
	MCP Integration	First-class Model Context Protocol support
	100+ Toolkits	Pre-built tools for data, web, and enterprise APIs
Runtime	Production Ready	FastAPI-based runtime with SSE endpoints
	Control Plane	Integrated UI for monitoring and debugging
	Multimodal	Process text, images, audio, video, files

Performance Benchmarks

Agno is optimized for performance:

- **Agent instantiation:** ~3µs (529× faster than Langgraph)
- **Memory footprint:** ~6.6KiB (24× lower than Langgraph)
- **Stateless & Scalable:** Horizontally scalable by design

---

2. Getting Started

Prerequisites

- Python 3.9 or higher
- An LLM provider API key (e.g., `OPENAI_API_KEY`)
- Basic understanding of Python

Installation

Create a virtual environment and install Agno:

```
# Create and activate virtual environment
python3 -m venv .venv
source .venv/bin/activate  # On Windows: .venv\Scripts\activate

# Install Agno and OpenAI
pip install -U agno openai
```

Your First Agent

Let's create your first AI agent in just a few lines:

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat

# Create an agent
agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    instructions="You are a helpful assistant",
    markdown=True,
)

# Chat with your agent
agent.print_response("Hello! What can you do?", stream=True)
```

What's happening here?

1. We import the `Agent` class and `OpenAIChat` model
2. We create an agent with:
    - A model (GPT-4o in this case)
    - Instructions that define its personality
    - `markdown=True` to get nicely formatted output
3. We send a message and stream the response

Adding Tools to Your Agent

Agents become powerful when they can use tools. Here's an agent that can search the web:

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.tools.duckduckgo import DuckDuckGoTools

# Create an agent with web search capability
agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[DuckDuckGoTools()],
    instructions="Search the web for current information",
    show_tool_calls=True,  # Show when tools are being used
    markdown=True,
)

# Ask a question that requires web search
agent.print_response(
    "What are the latest developments in AI this week?",
    stream=True
)
```

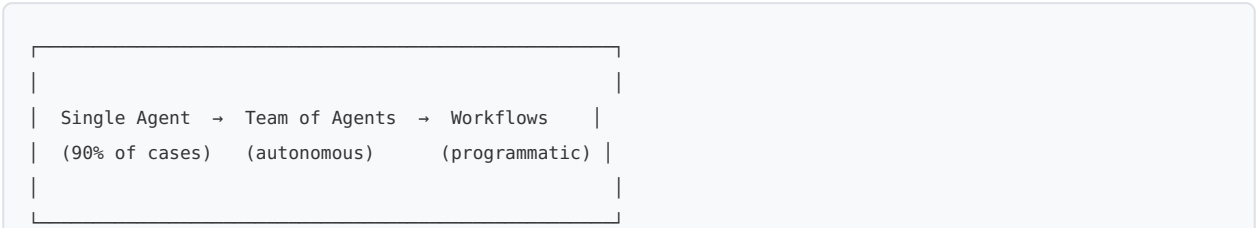
Key concept: Tools extend what your agent can do. Agno includes 100+ pre-built tools for: - Web search - APIs - Databases - File operations - And much more!

---

3. Core Concepts

The Three Building Blocks

Agno provides three main abstractions for building AI systems:



When to Use Each Pattern

Pattern	Best For	Example Use Case
Single Agent	One clear task or domain	Search assistant, code reviewer
Team	Multiple perspectives needed	Research + Analysis + Writing
Workflow	Sequential steps with control	Data pipeline, content creation

Critical Performance Rule: Agent Reuse

❌ **WRONG - Don't do this:**

```
# Creating agents in loops is SLOW
for query in queries:
    agent = Agent(...)  # Don't recreate!
    agent.run(query)
```

✅ CORRECT - Do this:

```
# Create once, reuse many times
agent = Agent(...)

for query in queries:
    agent.run(query)  # Fast!
```

Why? Agent creation has overhead. Reusing agents gives you 100-500× better performance.

---

4. Agents - The Foundation

What is an Agent?

An Agent is an AI assistant that can: - Have conversations - Use tools to take actions - Remember context across interactions - Access knowledge bases - Follow specific instructions

Basic Agent Structure

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat

agent = Agent(
    # Required
    model=OpenAIChat(id="gpt-4o"),

    # Optional but recommended
    name="My Assistant",
    instructions=["You are helpful", "Always cite sources"],
    markdown=True,

    # Tools
    tools=[],  # Add tools here

    # Memory & Context
    add_history_to_context=True,
    num_history_runs=3,

    # Knowledge
    knowledge=None,  # Add knowledge base
    search_knowledge=False,

    # Database
    db=None,  # Add for persistence
)
```

Agent with Structured Output

Get guaranteed JSON responses with Pydantic models:

```
from pydantic import BaseModel, Field
from agno.agent import Agent
from agno.models.openai import OpenAIChat

# Define your output structure
class MovieRecommendation(BaseModel):
    title: str = Field(description="Movie title")
    year: int = Field(description="Release year")
    genre: str = Field(description="Primary genre")
    reason: str = Field(description="Why this recommendation")

# Create agent with output schema
agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    output_schema=MovieRecommendation,
    instructions="Recommend movies based on user preferences"
)

# Get typed response
result: MovieRecommendation = agent.run(
    "Recommend a sci-fi movie from the 2020s"
).content

print(f"Title: {result.title}")
print(f"Year: {result.year}")
print(f"Reason: {result.reason}")
```

Why use structured output? - Type safety - Validation - Easy to integrate with your app - No parsing errors

Agent with Chat History

Remember conversations across sessions:

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.db.sqlite import SqliteDb

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),

    # Add database for persistence
    db=SqliteDb(db_file="tmp/agents.db"),

    # Add history to context
    add_history_to_context=True,
    num_history_runs=5,  # Include last 5 interactions

    # User identification
    user_id="user-123",
)

# First conversation
agent.print_response("My name is Alice", stream=True)

# Later conversation (agent remembers!)
agent.print_response("What's my name?", stream=True)
# Output: "Your name is Alice!"
```

Agent with Knowledge (RAG)

Give your agent a knowledge base to search:

```python
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.knowledge.knowledge import Knowledge
from agno.vectordb.lancedb import LanceDb, SearchType
from agno.knowledge.embedder.openai import OpenAIEmbedder

# Create knowledge base
knowledge = Knowledge(
    vector_db=LanceDb(
        uri="tmp/lancedb",
        table_name="knowledge_base",
        search_type=SearchType.hybrid,
        embedder=OpenAIEmbedder(id="text-embedding-3-small"),
    ),
)

# Load documents
knowledge.load_documents([
    "path/to/document1.pdf",
    "path/to/document2.txt",
])

# Create agent with knowledge
agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    knowledge=knowledge,
    search_knowledge=True,  # Critical: enables RAG
    instructions="Use the knowledge base and cite sources"
)

agent.print_response(
    "What does the documentation say about X?",
    stream=True
)
```

Key point: `search_knowledge=True` makes the agent automatically search the knowledge base when needed.

Custom Tools for Agents

Create your own tools as simple Python functions:

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat

def calculate_mortgage(
    principal: float,
    interest_rate: float,
    years: int
) -> str:
    """Calculate monthly mortgage payment.

    Args:
        principal: Loan amount in dollars
        interest_rate: Annual interest rate (e.g., 4.5 for 4.5%)
        years: Loan term in years

    Returns:
        Formatted string with monthly payment
    """
    monthly_rate = (interest_rate / 100) / 12
    num_payments = years * 12

    monthly_payment = principal * (
        monthly_rate * (1 + monthly_rate) ** num_payments
    ) / ((1 + monthly_rate) ** num_payments - 1)

    return f"Monthly payment: ${monthly_payment:.2f}"

# Create agent with custom tool
agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[calculate_mortgage],
    instructions="Help users with mortgage calculations",
    show_tool_calls=True,
)

agent.print_response(
    "What would my monthly payment be on a $300,000 loan at 4.5% for 30 years?",
    stream=True
)
```

How tools work: 1. The function docstring tells the agent what the tool does 2. Type hints tell the agent what parameters are needed 3. The agent decides when to use the tool 4. The agent interprets the result for the user

Async Agents

For high-performance applications, use async:

```
import asyncio
from agno.agent import Agent
from agno.models.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    instructions="You are a helpful assistant"
)

async def main():
    # Run multiple queries concurrently
    tasks = [
        agent.arun("What is Python?"),
        agent.arun("What is JavaScript?"),
        agent.arun("What is Rust?"),
    ]

    results = await asyncio.gather(*tasks)

    for result in results:
        print(result.content)

asyncio.run(main())
```

---

5. Teams - Multi-Agent Collaboration

What is a Team?

A Team is a group of specialized agents that work together autonomously. A team leader (powered by an LLM) decides which agents to use for each task.

When to Use Teams

Use teams when: - You need multiple perspectives (research + analysis + writing) - Tasks require different expertise - You want agents to collaborate autonomously - Complexity exceeds what one agent can handle

Basic Team Structure

```
from agno.agent import Agent
from agno.team.team import Team
from agno.models.openai import OpenAIChat
from agno.tools.duckduckgo import DuckDuckGoTools

# Create specialized agents
researcher = Agent(
    name="Researcher",
    role="Search the web for information",
    model=OpenAIChat(id="gpt-4o"),
    tools=[DuckDuckGoTools()],
    instructions=["Always find current information", "Cite sources"],
)

analyst = Agent(
    name="Analyst",
    role="Analyze data and provide insights",
    model=OpenAIChat(id="gpt-4o"),
    instructions=["Provide data-driven insights", "Be objective"],
)

writer = Agent(
    name="Writer",
    role="Write engaging content",
    model=OpenAIChat(id="gpt-4o"),
    instructions=["Write clearly and concisely", "Engage the reader"],
)

# Create the team
team = Team(
    name="Content Team",
    members=[researcher, analyst, writer],
    model=OpenAIChat(id="gpt-4o"),  # Leader uses this model
    instructions="Create comprehensive, well-researched content",
    markdown=True,
)

# Use the team
team.print_response(
    "Write an article about the future of renewable energy",
    stream=True
)
```

How it works: 1. The team leader receives the task 2. It decides which agents to involve 3. Agents collaborate and share results 4. The leader synthesizes a final response

Team with Shared Knowledge

Teams can share a knowledge base:

```
from agno.agent import Agent
from agno.team.team import Team
from agno.knowledge.knowledge import Knowledge
from agno.vectordb.pgvector import PgVector

# Create shared knowledge base
knowledge = Knowledge(
    vector_db=PgVector(
        table_name="company_docs",
        db_url="postgresql+psycopg://ai:ai@localhost:5532/ai"
    )
)

# Load company documents
knowledge.load_documents([
    "company_policies.pdf",
    "product_docs.pdf",
])

# Create agents with shared knowledge
support_agent = Agent(
    name="Support Agent",
    role="Answer customer questions",
    knowledge=knowledge,
    search_knowledge=True,
)

sales_agent = Agent(
    name="Sales Agent",
    role="Handle sales inquiries",
    knowledge=knowledge,
    search_knowledge=True,
)

# Create team
team = Team(
    name="Customer Team",
    members=[support_agent, sales_agent],
    knowledge=knowledge,  # Team also has access
)
```

Team with Persistence

Store team interactions in a database:

```
from agno.db.postgres import PostgresDb

db = PostgresDb(db_url="postgresql+psycopg://ai:ai@localhost:5532/ai")

team = Team(
    name="Research Team",
    members=[researcher, analyst, writer],
    model=OpenAIChat(id="gpt-4o"),

    # Add database
    db=db,

    # Add history
    add_history_to_context=True,
    num_history_runs=3,

    # User tracking
    user_id="team-user-123",
)
```

---

6. Workflows - Orchestration

What is a Workflow?

A Workflow is a series of steps executed in a specific order. Unlike Teams (where agents decide autonomously), Workflows give you programmatic control.

When to Use Workflows

Use workflows when: - You have clear, sequential steps - You need conditional logic or branching - You want full control over execution order - You're building data pipelines

Workflow Patterns

Agno Workflows 2.0 supports multiple patterns:

```
1. Sequential    → → →
2. Parallel      ⇒ ⇒ ⇒
3. Conditional   → ? →
4. Loops         → ↻ →
5. Routers       → ⊕ →
```

Basic Sequential Workflow

```
from agno.agent import Agent
from agno.workflow.workflow import Workflow
from agno.models.openai import OpenAIChat

# Create agents
researcher = Agent(
    name="Researcher",
    model=OpenAIChat(id="gpt-4o"),
    instructions="Research the topic thoroughly"
)

writer = Agent(
    name="Writer",
    model=OpenAIChat(id="gpt-4o"),
    instructions="Write based on research"
)

editor = Agent(
    name="Editor",
    model=OpenAIChat(id="gpt-4o"),
    instructions="Polish and improve the content"
)

# Create workflow
workflow = Workflow(
    name="Content Creation Pipeline",
    steps=[researcher, writer, editor],
)

# Run workflow
workflow.print_response(
    "Create an article about quantum computing",
    markdown=True
)
```

What happens: 1. Researcher investigates quantum computing 2. Writer creates content from research 3. Editor polishes the final result

Workflow with Functions

Mix agents with custom functions:

```
from agno.workflow import Step, Workflow
from agno.workflow.types import StepInput, StepOutput

def data_preprocessor(step_input: StepInput) -> StepOutput:
    """Custom preprocessing logic."""
    previous_content = step_input.previous_step_content or ""

    # Process the data
    processed = previous_content.upper()  # Example transformation

    return StepOutput(
        content=f"Processed: {processed}",
        success=True
    )

workflow = Workflow(
    name="Mixed Pipeline",
    steps=[
        researcher,          # Agent
        data_preprocessor,   # Function
        writer,              # Agent
    ]
)
```

Parallel Execution

Run steps concurrently for speed:

```
from agno.workflow import Parallel, Step, Workflow

# Different research sources
hn_researcher = Agent(name="HN Researcher", ...)
web_researcher = Agent(name="Web Researcher", ...)
academic_researcher = Agent(name="Academic Researcher", ...)

# Synthesizer combines results
synthesizer = Agent(name="Synthesizer", ...)

workflow = Workflow(
    name="Parallel Research",
    steps=[
        Parallel(
            Step(name="HackerNews", agent=hn_researcher),
            Step(name="Web", agent=web_researcher),
            Step(name="Academic", agent=academic_researcher),
        ),
        Step(name="Synthesis", agent=synthesizer),
    ]
)
```

Benefit: All three researchers work simultaneously, then the synthesizer combines their findings.

Conditional Workflows

Execute steps based on conditions:

```
from agno.workflow import Condition, Step, Workflow

def is_tech_topic(step_input) -> bool:
    """Check if topic is technology-related."""
    topic = step_input.input.lower()
    return any(kw in topic for kw in ["ai", "tech", "software"])

tech_expert = Agent(name="Tech Expert", ...)
general_expert = Agent(name="General Expert", ...)

workflow = Workflow(
    name="Conditional Research",
    steps=[
        Condition(
            name="Tech Check",
            evaluator=is_tech_topic,
            steps=[Step(name="Tech Research", agent=tech_expert)]
        ),
        Step(name="General Research", agent=general_expert),
    ]
)
```

Loop Workflows

Iterate until a condition is met:

```
from agno.workflow import Loop, Step, Workflow

def quality_check(outputs) -> bool:
    """Return True to stop, False to continue."""
    return any(len(output.content) > 500 for output in outputs)

workflow = Workflow(
    name="Quality-Driven Research",
    steps=[
        Loop(
            name="Research Loop",
            steps=[Step(name="Deep Research", agent=researcher)],
            end_condition=quality_check,
            max_iterations=3
        ),
        Step(name="Final Review", agent=editor),
    ]
)
```

Router (Dynamic Branching)

Route to different paths based on content:

```
from agno.workflow import Router, Step, Workflow
from typing import List

def route_by_topic(step_input) -> List[Step]:
    """Choose path based on topic."""
    topic = step_input.input.lower()

    if "tech" in topic:
        return [Step(name="Tech Path", agent=tech_expert)]
    elif "business" in topic:
        return [Step(name="Business Path", agent=biz_expert)]
    else:
        return [Step(name="General Path", agent=generalist)]

workflow = Workflow(
    name="Smart Routing",
    steps=[
        Router(
            name="Topic Router",
            selector=route_by_topic,
            choices=[tech_step, biz_step, general_step]
        ),
        Step(name="Synthesis", agent=synthesizer),
    ]
)
```

Complex Workflow Example

Combine multiple patterns:

```
workflow = Workflow(
    name="Advanced Pipeline",
    steps=[
        # Parallel research with conditions
        Parallel(
            Condition(
                evaluator=is_tech_topic,
                steps=[Step(agent=tech_researcher)]
            ),
            Condition(
                evaluator=is_business_topic,
                steps=[
                    Loop(
                        steps=[Step(agent=market_researcher)],
                        end_condition=quality_check,
                        max_iterations=3
                    )
                ]
            ),
        ),

        # Custom processing
        Step(executor=data_processor),

        # Route to appropriate format
        Router(
            selector=format_selector,
            choices=[blog_step, report_step, social_step]
        ),

        # Final review
        Step(agent=reviewer),
    ]
)
```

Workflow with State

Share data across workflow steps:

```
def add_to_cart(session_state, item: str) -> str:
    """Add item to shopping cart."""
    if "cart" not in session_state:
        session_state["cart"] = []
    session_state["cart"].append(item)
    return f"Added {item} to cart"

shopping_agent = Agent(
    name="Shopping Assistant",
    tools=[add_to_cart],
)

workflow = Workflow(
    name="Shopping Workflow",
    session_state={},  # Initialize shared state
    steps=[shopping_agent, checkout_agent],
)
```

---

7. Knowledge & RAG

What is Knowledge?

Knowledge in Agno is information that agents can search at runtime. This enables Retrieval-Augmented Generation (RAG), where agents:

1. Search relevant documents
2. Use retrieved information to answer questions
3. Cite sources for transparency

Why Use Knowledge?

- **Accuracy:** Ground responses in your documents
- **Up-to-date:** Always use latest information
- **Transparency:** Cite sources
- **Control:** Use your own data, not just pre-training

Basic Knowledge Setup

```
from agno.knowledge.knowledge import Knowledge
from agno.vectordb.pgvector import PgVector
from agno.knowledge.embedder.openai import OpenAIEmbedder

# Create knowledge base
knowledge = Knowledge(
    vector_db=PgVector(
        table_name="documents",
        db_url="postgresql+psycopg://ai:ai@localhost:5532/ai",
        embedder=OpenAIEmbedder(id="text-embedding-3-small"),
    )
)

# Load documents
knowledge.load_documents([
    "documentation.pdf",
    "user_manual.txt",
    "faq.md",
])
```

Supported Vector Databases

Agno supports 20+ vector databases:

Database	Use Case
PgVector	Production, PostgreSQL users
LanceDB	Fast local development
Pinecone	Fully managed cloud
Qdrant	Open-source, self-hosted
Weaviate	GraphQL interface
Chroma	Simple, lightweight
Milvus	Large-scale production

Adding Content to Knowledge

```
# From files
knowledge.load_documents([
    "path/to/file1.pdf",
    "path/to/file2.txt",
])

# From URLs
knowledge.add_content(
    url="https://example.com/document.pdf"
)

# From text directly
knowledge.add_content(
    content="This is important information...",
    name="Important Facts"
)
```

Search Types

```
from agno.vectordb.lancedb import SearchType

knowledge = Knowledge(
    vector_db=LanceDb(
        uri="tmp/lancedb",
        table_name="docs",
        search_type=SearchType.hybrid,  # Best for most cases
    )
)
```

Search types: - `SearchType.vector` : Semantic similarity - `SearchType.keyword` : Exact keyword match - `SearchType.hybrid` : Combines both (recommended)

Agent with Knowledge (Agentic RAG)

```
agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    knowledge=knowledge,
    search_knowledge=True,  # CRITICAL: enables automatic search
    instructions=[
        "Use the knowledge base to answer questions",
        "Always cite sources",
        "If information is not in the knowledge base, say so"
    ]
)

agent.print_response(
    "What is our return policy?",
    stream=True
)
```

Key difference from traditional RAG: - **Traditional RAG:** Always retrieves, even when not needed - **Agentic RAG:** Agent decides when to search (more efficient)

Reranking for Better Results

Improve search quality with reranking:

```
from agno.reranker.cohere import CohereReranker

knowledge = Knowledge(
    vector_db=PgVector(...),
    reranker=CohereReranker(),
    num_documents=5,  # Retrieve 5 documents
    rerank_results=3,  # Return top 3 after reranking
)
```

Knowledge with Metadata Filtering

Filter documents by metadata:

```
# Add documents with metadata
knowledge.add_content(
    content="Product documentation...",
    name="Product Docs",
    metadata={"category": "products", "version": "2.0"}
)

# Search with filters
agent.run(
    "What are the product features?",
    filters={"category": "products"}
)
```

8. Memory System

What is Memory?

Memory allows agents to remember facts and insights about users across conversations. Think of it as giving your agent "ChatGPT-like memory."

When to Use Memory

- Personalize responses for different users
- Remember preferences and context
- Build long-term relationships with users
- Avoid asking the same questions repeatedly

Basic Memory Setup

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.db.postgres import PostgresDb

db = PostgresDb(db_url="postgresql+psycopg://ai:ai@localhost:5532/ai")

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    db=db,

    # Enable memory
    enable_user_memories=True,

    # User identification
    user_id="user-123",

    instructions=[
        "Learn about the user through conversation",
        "Remember important details",
        "Use memories to personalize responses"
    ]
)
```

How Memory Works

```
# First conversation
agent.print_response("My name is Alice and I love Python")
# Agent stores: "User's name is Alice, enjoys Python programming"

# Second conversation (later)
agent.print_response("What programming languages do I like?")
# Agent retrieves memory and responds: "You mentioned you love Python!"
```

Manual Memory Management

```
from agno.memory.memory import Memory

memory = Memory(
    db=db,
    user_id="user-123",
)

# Add a memory
memory.add_memory("User prefers morning meetings")

# Get all memories
memories = memory.get_memories()

# Update a memory
memory.update_memory(
    memory_id="mem-123",
    content="User prefers afternoon meetings"
)

# Delete a memory
memory.delete_memory(memory_id="mem-123")
```

Memory with Classification

Automatically classify memories:

```
agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    db=db,
    enable_user_memories=True,

    # Classify memories
    memory_classification=[
        "personal_info",
        "preferences",
        "work_related",
        "hobbies"
    ],
)
```

---

9. Tools & Integrations

Built-in Toolkits

Agno provides 100+ pre-built tools organized into toolkits:

Web & Search Tools

```
from agno.tools.duckduckgo import DuckDuckGoTools
from agno.tools.serp import SerpTools
from agno.tools.exa import ExaTools

agent = Agent(
    tools=[
        DuckDuckGoTools(),    # Web search
        SerpTools(),          # Google search
        ExaTools(),           # AI-powered search
    ]
)
```

Data & Finance Tools

```
from agno.tools.yfinance import YFinanceTools
from agno.tools.pandas import PandasTools

agent = Agent(
    tools=[
        YFinanceTools(
            stock_price=True,
            company_info=True,
            analyst_recommendations=True,
        ),
        PandasTools(),  # Data analysis
    ]
)
```

Code & Development Tools

```
from agno.tools.python import PythonTools
from agno.tools.shell import ShellTools

agent = Agent(
    tools=[
        PythonTools(),  # Execute Python code
        ShellTools(),   # Run shell commands
    ]
)
```

Communication Tools

```
from agno.tools.email import EmailTools
from agno.tools.slack import SlackTools

agent = Agent(
    tools=[
        EmailTools(),
        SlackTools(token="your-slack-token"),
    ]
)
```

Creating Custom Tools

Simple Function Tool

```
def get_weather(city: str) -> str:
    """Get current weather for a city.

    Args:
        city: Name of the city

    Returns:
        Weather description
    """
    # Your weather API call here
    return f"The weather in {city} is sunny, 72°F"

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[get_weather],
)
```

Tool with Multiple Parameters

```
def book_meeting(
    title: str,
    date: str,
    attendees: list[str],
    duration_minutes: int = 30
) -> str:
    """Book a meeting.

    Args:
        title: Meeting title
        date: Date in YYYY-MM-DD format
        attendees: List of attendee emails
        duration_minutes: Meeting duration (default: 30)

    Returns:
        Confirmation message
    """
    return f"Meeting '{title}' booked for {date} with {len(attendees)} attendees"

agent = Agent(
    tools=[book_meeting],
)
```

Custom Toolkit Class

For complex integrations:

```
from agno.tools.toolkit import Toolkit

class CustomerToolkit(Toolkit):
    def __init__(self):
        super().__init__(name="customer_toolkit")

        self.register(self.get_customer_info)
        self.register(self.update_customer)

    def get_customer_info(self, customer_id: str) -> dict:
        """Get customer information.

        Args:
            customer_id: Customer ID

        Returns:
            Customer data
        """
        # Your database query here
        return {"id": customer_id, "name": "John Doe"}

    def update_customer(self, customer_id: str, **updates) -> str:
        """Update customer information.

        Args:
            customer_id: Customer ID
            **updates: Fields to update

        Returns:
            Success message
        """
        # Your update logic here
        return f"Customer {customer_id} updated"

agent = Agent(
    tools=[CustomerToolkit()],
)
```

Model Context Protocol (MCP)

Integrate with MCP servers:

```
from agno.tools.mcp import MCPTools

agent = Agent(
    tools=[
        MCPTools(
            transport="streamable-http",
            url="https://docs.agno.com/mcp"
        )
    ]
)
```

10. AgentOS - Production Runtime

What is AgentOS?

AgentOS is a production-ready FastAPI application for serving your agents, teams, and workflows. It provides:

- RESTful API endpoints
- WebSocket support for streaming
- Database integration
- Monitoring and metrics
- Control plane UI

Basic AgentOS Setup

```
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.os import AgentOS
from agno.db.postgres import PostgresDb

# Create your agents
assistant = Agent(
    name="Assistant",
    model=OpenAIChat(id="gpt-4o"),
    db=PostgresDb(db_url="postgresql+psycopg://..."),
)

# Create AgentOS
agent_os = AgentOS(
    id="my-agent-os",
    description="My production AgentOS",
    agents=[assistant],
)

# Get the FastAPI app
app = agent_os.get_app()

# Serve the app
if __name__ == "__main__":
    agent_os.serve(app="my_os:app", reload=True)
```

Running AgentOS

```
# Start the server
python my_os.py

# Access points:
# - App: http://localhost:7777
# - API Docs: http://localhost:7777/docs
# - Config: http://localhost:7777/config
```

AgentOS with Multiple Agents

```
agent_os = AgentOS(
    agents=[
        web_agent,
        finance_agent,
        support_agent,
    ],
    teams=[
        research_team,
    ],
    workflows=[
        content_workflow,
    ],
)
```

Using the AgentOS API

```
# Create a new run
curl -X POST "http://localhost:7777/agents/assistant/runs" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "message=Hello&stream=true&user_id=user-123"

# Get session history
curl "http://localhost:7777/sessions?agent_id=assistant"

# Get memories
curl "http://localhost:7777/memories?user_id=user-123"
```

AgentOS Security

Secure your AgentOS with bearer token authentication:

```
agent_os = AgentOS(
    agents=[assistant],
    security_key="your-secret-key-here",
)
```

Then include the token in requests:

```
curl -H "Authorization: Bearer your-secret-key-here" \
  http://localhost:7777/agents/assistant/runs
```

Custom FastAPI Integration

Bring your own FastAPI app:

```
from fastapi import FastAPI

# Your custom app
app = FastAPI(title="My Custom App")

@app.get("/status")
async def status():
    return {"status": "healthy"}

# Add AgentOS to your app
agent_os = AgentOS(
    agents=[assistant],
    base_app=app,  # Your app
)

app = agent_os.get_app()
```

AgentOS Control Plane

Connect to the web UI at `https://os.agno.com`:

1. Sign in to Agno
2. Click "Connect OS"
3. Enter your endpoint: `http://localhost:7777`
4. Start using the UI!

The control plane provides: - Chat interface - Session management - Knowledge base management - Memory viewer - Metrics and monitoring - User management

---

11. Database Integration

Supported Databases

Agno supports multiple databases for different needs:

Database	Best For	Production Ready
PostgreSQL	Production	✅ Yes
SQLite	Development	❌ Dev only
MongoDB	Document storage	✅ Yes
MySQL	MySQL users	✅ Yes
Redis	Cache/session	✅ Yes
DynamoDB	AWS users	✅ Yes
SingleStore	High performance	✅ Yes
Firestore	Firebase users	✅ Yes

PostgreSQL (Recommended for Production)

```
from agno.db.postgres import PostgresDb

db = PostgresDb(
    db_url="postgresql+psycopg://user:pass@localhost:5432/db",
    table_name="agent_sessions",
)

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    db=db,
)
```

SQLite (Development Only)

```
from agno.db.sqlite import SqliteDb

db = SqliteDb(
    db_file="tmp/agents.db",
    table_name="agent_sessions",
)

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    db=db,
)
```

MongoDB

```
from agno.db.mongodb import MongoDb

db = MongoDb(
    connection_string="mongodb://localhost:27017",
    db_name="agno",
    collection_name="sessions",
)
```

What Gets Stored?

When you add a database to an agent:

- **Sessions:** Conversation history
- **Messages:** All messages exchanged
- **State:** Session state data
- **Memories:** User memories (if enabled)
- **Metrics:** Performance data

---

12. Cookbook Examples

Getting Started Examples

1. Basic Agent

```
# cookbook/getting_started/01_basic_agent.py
from agno.agent import Agent
from agno.models.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    instructions="You are an enthusiastic news reporter! 🗽",
    markdown=True,
)

agent.print_response(
    "Tell me about a breaking news story",
    stream=True
)
```

2. Agent with Tools

```
# cookbook/getting_started/02_agent_with_tools.py
from agno.agent import Agent
from agno.models.openai import OpenAIChat
from agno.tools.duckduckgo import DuckDuckGoTools

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[DuckDuckGoTools()],
    instructions="Search the web for current information",
    show_tool_calls=True,
)

agent.print_response(
    "What are the latest AI developments?",
    stream=True
)
```

3. Agent with Knowledge

```
# cookbook/getting_started/03_agent_with_knowledge.py
from agno.agent import Agent
from agno.knowledge.knowledge import Knowledge
from agno.vectordb.lancedb import LanceDb

knowledge = Knowledge(
    vector_db=LanceDb(uri="tmp/lancedb", table_name="recipes")
)

knowledge.load_documents(["recipes.pdf"])

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    knowledge=knowledge,
    search_knowledge=True,
    instructions="Answer questions using the recipe knowledge base",
)

agent.print_response("How do I make Thai curry?", stream=True)
```

4. Agent with Memory

```
# cookbook/getting_started/04_agent_with_memory.py
from agno.agent import Agent
from agno.db.sqlite import SqliteDb

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    db=SqliteDb(db_file="tmp/agent.db"),
    enable_user_memories=True,
    user_id="user-123",
    instructions="Learn about the user and personalize responses",
)

# First conversation
agent.print_response("My favorite color is blue", stream=True)

# Later conversation
agent.print_response("What's my favorite color?", stream=True)
```

Agent Examples

Async Agents

```
# cookbook/agents/async/basic.py
import asyncio
from agno.agent import Agent
from agno.models.openai import OpenAIChat

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    instructions="You are a helpful assistant"
)

async def main():
    # Run multiple queries concurrently
    results = await asyncio.gather(
        agent.arun("What is Python?"),
        agent.arun("What is JavaScript?"),
        agent.arun("What is Rust?"),
    )

    for result in results:
        print(f"\n{result.content}\n")

asyncio.run(main())
```

Streaming Events

```
# cookbook/agents/events/basic_agent_events.py
from agno.agent import Agent
from agno.run.response import RunEvent

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[DuckDuckGoTools()],
)

for event in agent.run("Search for AI news", stream=True):
    if event.event == RunEvent.tool_call_started:
        print(f"🔧 Calling tool: {event.tool_name}")
    elif event.event == RunEvent.tool_call_completed:
        print(f"✅ Tool completed: {event.tool_name}")
    elif event.event == RunEvent.run_completed:
        print(f"📝 Final response: {event.content}")
```

Human-in-the-Loop

```
# cookbook/agents/human_in_the_loop/confirmation_required.py
from agno.agent import Agent
from agno.tools import toolkit

def send_email(to: str, subject: str, body: str) -> str:
    """Send an email (requires user confirmation)."""
    return f"Email sent to {to}"

# Mark tool as requiring confirmation
send_email.agno_confirmation_required = True

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[send_email],
    instructions="Help users send emails",
)

# Agent will pause and ask for confirmation before sending
response = agent.run("Send an email to john@example.com")

if response.needs_confirmation():
    print("Confirmation needed:")
    print(response.confirmation_message)

    # User approves
    response = agent.run(confirm=True)
```

Team Examples

Basic Team

```
# cookbook/teams/basic_flows/01_basic_team.py
from agno.agent import Agent
from agno.team.team import Team

researcher = Agent(
    name="Researcher",
    role="Research information",
    tools=[DuckDuckGoTools()],
)

writer = Agent(
    name="Writer",
    role="Write content",
)

team = Team(
    name="Content Team",
    members=[researcher, writer],
    model=OpenAIChat(id="gpt-4o"),
)

team.print_response(
    "Write an article about renewable energy",
    stream=True
)
```

Team with Shared Knowledge

```
# cookbook/teams/knowledge/shared_knowledge.py
knowledge = Knowledge(
    vector_db=PgVector(table_name="company_docs", ...)
)

agent1 = Agent(name="Support", knowledge=knowledge, search_knowledge=True)
agent2 = Agent(name="Sales", knowledge=knowledge, search_knowledge=True)

team = Team(
    members=[agent1, agent2],
    knowledge=knowledge,  # Team also searches
)
```

Workflow Examples

Sequential Workflow

```
# cookbook/workflows/_01_basic_workflows/_01_sequence_of_steps/sync/sequence_of_steps.py
from agno.workflow import Step, Workflow

workflow = Workflow(
    name="Content Pipeline",
    steps=[
        Step(name="Research", agent=researcher),
        Step(name="Write", agent=writer),
        Step(name="Edit", agent=editor),
    ]
)

workflow.print_response(
    "Create an article about AI",
    markdown=True
)
```

Parallel Workflow

```
# cookbook/workflows/_04_workflows_parallel_execution/sync/parallel_steps_workflow.py
from agno.workflow import Parallel, Step

workflow = Workflow(
    name="Parallel Research",
    steps=[
        Parallel(
            Step(name="HN", agent=hn_researcher),
            Step(name="Web", agent=web_researcher),
            Step(name="Academic", agent=academic_researcher),
        ),
        Step(name="Synthesis", agent=synthesizer),
    ]
)
```

Conditional Workflow

```
# cookbook/workflows/_02_workflows_conditional_execution/sync/condition_with_list_of_steps.py
from agno.workflow import Condition, Step

def is_urgent(step_input) -> bool:
    return "urgent" in step_input.input.lower()

workflow = Workflow(
    steps=[
        Condition(
            evaluator=is_urgent,
            steps=[Step(agent=urgent_handler)]
        ),
        Step(agent=regular_handler),
    ]
)
```

Knowledge Examples

Loading Documents

```
# cookbook/knowledge/basic_operations/from_path.py
knowledge = Knowledge(vector_db=LanceDb(...))

# Load from files
knowledge.load_documents([
    "document1.pdf",
    "document2.txt",
    "document3.md",
])
```

Loading from URLs

```
# cookbook/knowledge/basic_operations/from_url.py
knowledge.add_content(
    url="https://example.com/document.pdf",
    name="Important Document"
)
```

Custom Chunking

```
# cookbook/knowledge/chunking/custom_chunk_size.py
from agno.knowledge.chunking import FixedSizeChunking

knowledge = Knowledge(
    vector_db=LanceDb(...),
    chunking_strategy=FixedSizeChunking(
        chunk_size=500,
        overlap=50
    )
)
```

Model Examples

Agno supports 40+ LLM providers. Here are some examples:

OpenAI

```
# cookbook/models/openai/basic.py
from agno.models.openai import OpenAIChat

agent = Agent(model=OpenAIChat(id="gpt-4o"))
```

Anthropic Claude

```
# cookbook/models/anthropic/basic.py
from agno.models.anthropic import Claude

agent = Agent(model=Claude(id="claude-sonnet-4-0"))
```

Google Gemini

```
# cookbook/models/google/gemini/basic.py
from agno.models.google import Gemini

agent = Agent(model=Gemini(id="gemini-2.0-flash-exp"))
```

AWS Bedrock

```
# cookbook/models/aws/bedrock/basic.py
from agno.models.aws import AwsBedrock

agent = Agent(model=AwsBedrock(id="anthropic.claude-3-sonnet-20240229-v1:0"))
```

Ollama (Local Models)

```
# cookbook/models/ollama/basic.py
from agno.models.ollama import Ollama

agent = Agent(model=Ollama(id="llama3"))
```

Reasoning Examples

Enable Reasoning

```
# cookbook/reasoning/basic_reasoning.py
agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    reasoning=True,  # Enable reasoning
    instructions="Think step by step",
)

agent.print_response(
    "Solve this logic puzzle: If all bloops are razzies and all razzies are lazzies, are all bloops definitely lazzies?",
    stream=True
)
```

---

13. Best Practices

Performance Best Practices

1. Reuse Agents (Critical!)

```
# ❌ WRONG - Creates agent every time (slow!)
def process_queries(queries):
    for query in queries:
        agent = Agent(...)  # Don't do this!
        agent.run(query)

# ✅ CORRECT - Create once, reuse (fast!)
def process_queries(queries):
    agent = Agent(...)  # Create once
    for query in queries:
        agent.run(query)  # Reuse
```

2. Use Async for Concurrency

```
# Process multiple requests concurrently
async def process_batch(queries):
    agent = Agent(...)
    tasks = [agent.arun(q) for q in queries]
    return await asyncio.gather(*tasks)
```

3. Use Appropriate Models

```
# Use cheaper models for simple tasks
simple_agent = Agent(model=OpenAIChat(id="gpt-4o-mini"))

# Use powerful models for complex reasoning
complex_agent = Agent(model=OpenAIChat(id="gpt-4o"))
```

Architecture Best Practices

1. Start with Single Agent

```
# Start simple
agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    tools=[DuckDuckGoTools()],
    instructions="You are a helpful assistant"
)

# Only add complexity when needed
# ❌ Don't start with teams/workflows unless necessary
```

2. Use Teams for Autonomy

```
# Use teams when agents should decide autonomously
team = Team(
    members=[researcher, analyst, writer],
    instructions="Collaborate to create comprehensive content"
)
```

3. Use Workflows for Control

```
# Use workflows when you need explicit control
workflow = Workflow(
    steps=[
        validate_input,  # You control the order
        process_data,
        generate_output,
    ]
)
```

Database Best Practices

1. PostgreSQL in Production

```
# ✅ Production
db = PostgresDb(db_url=os.getenv("DATABASE_URL"))

# ❌ Don't use SQLite in production
# db = SqliteDb(db_file="data.db")  # Dev only!
```

2. Connection Pooling

```
db = PostgresDb(
    db_url="postgresql+psycopg://...",
    pool_size=10,
    max_overflow=20,
)
```

3. Separate Databases

```
# Different databases for different concerns
agent_db = PostgresDb(...)          # Agent sessions
knowledge_db = PgVector(...)        # Knowledge base
metrics_db = PostgresDb(...)        # Metrics
```

Security Best Practices

1. Secure AgentOS

```
agent_os = AgentOS(
    agents=[...],
    security_key=os.getenv("AGENTOS_SECURITY_KEY"),  # From env
)
```

2. Validate Tool Inputs

```
def sensitive_operation(param: str) -> str:
    """Perform sensitive operation."""
    # Validate input
    if not is_valid(param):
        raise ValueError("Invalid input")

    # Proceed
    return perform_operation(param)
```

3. Use Guardrails

```
from agno.guardrails import Guardrails

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    guardrails=Guardrails(
        prevent_pii=True,
        prevent_prompt_injection=True,
        moderation=True,
    )
)
```

Knowledge Best Practices

1. Organize by Domain

```
# Separate knowledge bases for different domains
product_knowledge = Knowledge(...)
support_knowledge = Knowledge(...)
legal_knowledge = Knowledge(...)
```

2. Use Appropriate Search Type

```
# Hybrid search (recommended for most cases)
knowledge = Knowledge(
    vector_db=PgVector(...),
    search_type=SearchType.hybrid
)
```

3. Enable Reranking

```
from agno.reranker.cohere import CohereReranker

knowledge = Knowledge(
    vector_db=PgVector(...),
    reranker=CohereReranker(),
    num_documents=10,
    rerank_results=3,  # Return top 3
)
```

Error Handling

```
try:
    response = agent.run(user_input)

    if response.content:
        print(response.content)
    else:
        print("No response generated")

except Exception as e:
    logger.error(f"Agent error: {e}")
    # Handle error appropriately
```

Monitoring and Logging

```
import logging

# Configure logging
logging.basicConfig(level=logging.INFO)

agent = Agent(
    model=OpenAIChat(id="gpt-4o"),
    debug_mode=False,  # Disable in production
    monitoring=True,   # Enable metrics
)
```

---

14. API Reference

Agent Class

```
from agno.agent import Agent

agent = Agent(
    # Model (required)
    model: Model,

    # Identity
    id: Optional[str] = None,
    name: Optional[str] = None,
    role: Optional[str] = None,

    # Instructions
    instructions: Optional[List[str]] = None,

    # Tools
    tools: Optional[List[Union[Callable, Toolkit]]] = None,
    show_tool_calls: bool = False,

    # Knowledge
    knowledge: Optional[Knowledge] = None,
    search_knowledge: bool = False,

    # Memory
    enable_user_memories: bool = False,
    memory_classification: Optional[List[str]] = None,

    # Database
    db: Optional[Database] = None,

    # Context
    add_history_to_context: bool = False,
    num_history_runs: int = 3,
    add_datetime_to_context: bool = False,

    # Output
    output_schema: Optional[Type[BaseModel]] = None,
    markdown: bool = False,

    # User
    user_id: Optional[str] = None,
    session_id: Optional[str] = None,

    # Advanced
    reasoning: bool = False,
    debug_mode: bool = False,
)
```

Agent Methods

```
# Synchronous run
response = agent.run(
    message: str,
    stream: bool = False,
    session_id: Optional[str] = None,
    user_id: Optional[str] = None,
)

# Async run
response = await agent.arun(
    message: str,
    stream: bool = False,
)

# Print response
agent.print_response(
    message: str,
    stream: bool = True,
    markdown: bool = False,
)

# Get session
session = agent.get_session()

# Get memory
memories = agent.get_memories(user_id="user-123")
```

Team Class

```
from agno.team.team import Team

team = Team(
    # Members (required)
    members: List[Union[Agent, Team]],

    # Model for leader
    model: Model,

    # Identity
    id: Optional[str] = None,
    name: Optional[str] = None,

    # Instructions
    instructions: Optional[List[str]] = None,

    # Knowledge
    knowledge: Optional[Knowledge] = None,

    # Database
    db: Optional[Database] = None,

    # Context
    add_history_to_context: bool = False,
    num_history_runs: int = 3,

    # User
    user_id: Optional[str] = None,
)
```

Workflow Class

```
from agno.workflow.workflow import Workflow

workflow = Workflow(
    # Identity
    name: str,
    description: Optional[str] = None,

    # Steps (required)
    steps: Union[List[Step], Callable],

    # Database
    db: Optional[Database] = None,

    # State
    session_state: Optional[Dict] = None,

    # Events
    store_events: bool = False,
    events_to_skip: Optional[List[WorkflowRunEvent]] = None,
)
```

Knowledge Class

```
from agno.knowledge.knowledge import Knowledge

knowledge = Knowledge(
    # Vector DB (required)
    vector_db: VectorDB,

    # Embedder
    embedder: Optional[Embedder] = None,

    # Search
    num_documents: int = 5,
    search_type: SearchType = SearchType.hybrid,

    # Reranking
    reranker: Optional[Reranker] = None,
    rerank_results: Optional[int] = None,

    # Chunking
    chunking_strategy: Optional[ChunkingStrategy] = None,
)
```

Common Models

```
# OpenAI
from agno.models.openai import OpenAIChat
model = OpenAIChat(id="gpt-4o")

# Anthropic
from agno.models.anthropic import Claude
model = Claude(id="claude-sonnet-4-0")

# Google
from agno.models.google import Gemini
model = Gemini(id="gemini-2.0-flash-exp")

# OpenAI Azure
from agno.models.azure import AzureOpenAIChat
model = AzureOpenAIChat(id="gpt-4o")

# AWS Bedrock
from agno.models.aws import AwsBedrock
model = AwsBedrock(id="anthropic.claude-3-sonnet-20240229-v1:0")

# Ollama (Local)
from agno.models.ollama import Ollama
model = Ollama(id="llama3")
```

Common Databases

```
# PostgreSQL (Production)
from agno.db.postgres import PostgresDb
db = PostgresDb(db_url="postgresql+psycopg://...")

# SQLite (Development)
from agno.db.sqlite import SqliteDb
db = SqliteDb(db_file="tmp/db.db")

# MongoDB
from agno.db.mongodb import MongoDb
db = MongoDb(connection_string="mongodb://...")

# Redis
from agno.db.redis import RedisDb
db = RedisDb(host="localhost", port=6379)
```

Common Vector Databases

```
# PgVector (PostgreSQL)
from agno.vectordb.pgvector import PgVector
vector_db = PgVector(
    table_name="vectors",
    db_url="postgresql+psycopg://...",
)

# LanceDB
from agno.vectordb.lancedb import LanceDb
vector_db = LanceDb(
    uri="tmp/lancedb",
    table_name="vectors",
)

# Pinecone
from agno.vectordb.pinecone import PineconeDB
vector_db = PineconeDB(
    index_name="my-index",
    api_key="your-api-key",
)

# Qdrant
from agno.vectordb.qdrant import Qdrant
vector_db = Qdrant(
    collection="vectors",
    url="http://localhost:6333",
)
```

Conclusion

Quick Reference Card

```
# Basic Agent
agent = Agent(model=OpenAIChat(id="gpt-4o"))
agent.print_response("Hello!")

# Agent with Tools
agent = Agent(tools=[DuckDuckGoTools()])

# Agent with Knowledge
agent = Agent(knowledge=knowledge, search_knowledge=True)

# Agent with Memory
agent = Agent(db=db, enable_user_memories=True)

# Team
team = Team(members=[agent1, agent2], model=...)

# Workflow
workflow = Workflow(steps=[step1, step2, step3])

# AgentOS
agent_os = AgentOS(agents=[agent])
app = agent_os.get_app()
```

Next Steps

1. **Explore Examples:** Check out the `/workspace/cookbook` directory
2. **Read Docs:** Visit https://docs.agno.com
3. **Join Community:** https://community.agno.com
4. **Get Help:** https://discord.gg/4MtYHHrgA8

Performance Reminders

- ✅ Always reuse agents (don't create in loops)
- ✅ Use PostgreSQL in production (not SQLite)
- ✅ Start simple, add complexity as needed
- ✅ Use async for concurrent operations
- ✅ Enable `search_knowledge=True` for RAG

Common Patterns

```
# Research + Write + Edit
workflow = Workflow(steps=[researcher, writer, editor])

# Multi-source Research
workflow = Workflow(steps=[
    Parallel(hn_agent, web_agent, academic_agent),
    synthesizer
])

# Conditional Processing
workflow = Workflow(steps=[
    Condition(evaluator=check, steps=[special_agent]),
    regular_agent
])
```

Happy Building with Agno! 🚀

For more information: - Documentation: https://docs.agno.com - GitHub: https://github.com/agno-agi/agno - Community: https://community.agno.com - Discord: https://discord.gg/4MtYHHrgA8